

Project 2 — AegisFlow: Real-Time Transaction Risk & AML Investigation Platform

1. Project Title

AegisFlow — Real-Time Payment Fraud Scoring and Agentic Anti-Money-Laundering Investigation Platform

2. Executive Summary

AegisFlow is a financial-crime platform with two tightly integrated halves:

1. A **real-time fraud scoring engine**: every incoming payment is scored in under 100 ms using gradient-boosted models fed by an **online/offline feature store** (Redis online, PostgreSQL offline) with velocity features, behavioral profiles, and **graph features** (network proximity to known-fraud entities). Every score ships with SHAP-based reason codes.
2. An **agentic AML investigation workbench**: flagged cases enter a queue where a **multi-agent LLM team** (link analyst, pattern analyst, adverse-media researcher, narrative writer, QA reviewer) assembles evidence and drafts a regulator-style **Suspicious Activity Report (SAR) narrative** — always reviewed and approved by a human compliance analyst.

The system is fully buildable with public/synthetic data: **IEEE-CIS Fraud Detection** (Kaggle) for supervised training, plus the **AMLSim / SAML-D style synthetic transaction generator** included in the project for streaming, laundering typologies (smurfing, layering, round-tripping), and entity graphs.

3. Industry Domain

Finance — payments, digital banking, fintech risk & compliance.

4. Problem Statement

Global fraud losses exceed \$40B/year, and banks spend billions more on AML compliance. The operational reality is broken at both ends:

- **Detection**: legacy rule engines ("flag if amount > ₹50,000 and new device") produce 90%+ false positives, block legitimate customers (costing revenue and trust), and are trivially learned and evaded by fraud rings. Rules cannot see *networks* — mule accounts look individually innocent.
- **Investigation**: every flagged case requires an analyst to manually pull transaction history, map counterparties, search adverse media, and hand-write a SAR narrative — 2 to 6 hours per case. Backlogs run to weeks; quality is inconsistent; regulators penalize late or poor filings.

Point ML solutions exist but typically stop at a score with no explanation (unusable for compliance, where every decision must be justified) and no help with the investigation itself. AegisFlow attacks both:

explainable ML for the millisecond decision, and agentic AI for the hours-long investigation — with the human analyst kept firmly in control.

5. Business Objectives

1. Score transactions with p95 end-to-end latency < 100 ms (API receipt → decision).
2. Beat a published rules baseline by $\geq 30\%$ recall at 1% false-positive rate on the IEEE-CIS test split.
3. Every decision carries top-5 SHAP reason codes (100% explainability coverage).
4. Detect $\geq 80\%$ of injected laundering typologies (smurfing, layering, round-tripping) in the synthetic stream via graph + velocity features.
5. Reduce simulated investigation time from hours to < 15 minutes by auto-assembling evidence packs and draft narratives.
6. 100% of SAR drafts pass through human approval with full audit trail (zero autonomous filings).

6. Target Users

- **Fraud operations analysts** — triage real-time declines/reviews, tune thresholds.
- **AML compliance analysts** — work the case queue, review agent-assembled evidence, edit and approve SAR narratives.
- **Compliance managers / MLRO** — dashboards on case throughput, SLA, filing quality; final SAR sign-off.
- **Risk data scientists (you)** — own models, feature store, monitoring.
- **Customers (indirect)** — fewer false declines.

7. Complete Functional Requirements

Transaction ingestion & feature store - FR-1: Synthetic transaction generator streams realistic payment traffic (accounts, devices, merchants, IPs) with injected fraud rings and laundering typologies, labels retained for evaluation. - FR-2: Ingestion API accepts transactions, validates schema, persists to PostgreSQL, and updates online features in Redis atomically (velocity counters via sliding windows, behavioral aggregates). - FR-3: Offline feature pipeline (Pandas/PyArrow batch jobs) computes training features into point-in-time-correct feature tables; **the same feature definitions** (single Python module) drive both online and offline computation to eliminate training/serving skew. - FR-4: Entity graph maintained in PostgreSQL (edge tables) — accounts↔devices↔IPs↔merchants; graph feature job computes connected-component size, shortest-path-to-known-fraud, PageRank-style risk propagation (NetworkX batch, cached to Redis).

Real-time decisioning - FR-5: Scoring service returns `approve / review / decline` with calibrated probability, reason codes, and rule hits in < 100 ms p95. - FR-6: Hybrid decisioning: ML score + deterministic rule layer (sanctions-list hit, impossible-travel velocity) — rules can force decline regardless of score. - FR-7: Champion/challenger: challenger model scores in shadow; disagreement and lift tracked. - FR-8: Threshold management API with cost-based tuning (expected loss vs. friction) and instant rollback.

Case management & AML - FR-9: Case engine groups related alerts (same entity/ring) into cases with SLA timers and priority scoring. - FR-10: Multi-agent investigation pipeline per case: link analysis, temporal pattern analysis, adverse-media search over a bundled news corpus (RAG), narrative drafting, QA review

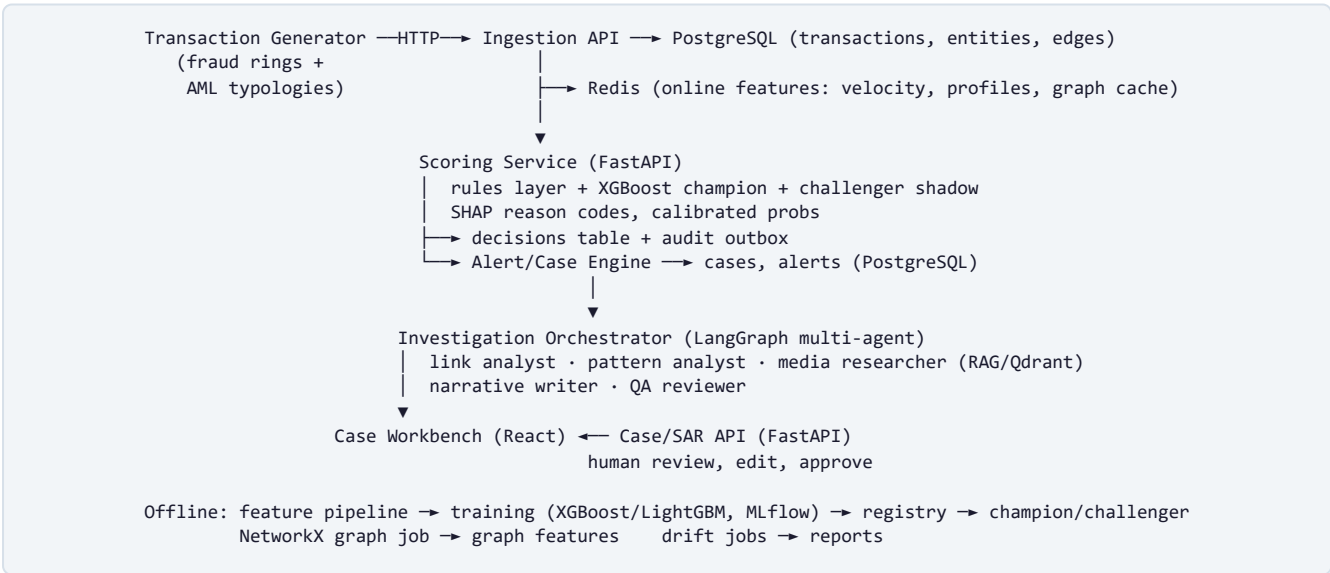
(see §15). - FR-11: Evidence pack: entity graph visualization data, transaction timeline, typology matches, media excerpts — every item traceable to source records. - FR-12: SAR narrative editor: analyst edits draft, every change versioned; approve/reject with mandatory rationale; export to FinCEN-style structured format. - FR-13: Feedback loop: analyst dispositions (fraud / not fraud) flow back as labels for scheduled retraining.

Platform - FR-14: JWT auth with roles (fraud_analyst, aml_analyst, manager, admin); case-level access control. - FR-15: React + FastAPI console: live decision stream, case queue, case workspace (graph view, timeline, narrative editor), model & drift dashboards. - FR-16: Full immutable audit log (decisions, threshold changes, case actions, SAR approvals). - FR-17: Prometheus metrics: latency histograms, decision mix, feature-store hit rate, drift stats.

8. Non-Functional Requirements

- **Performance:** scoring p95 < 100 ms, p99 < 150 ms at 300 TPS (load-tested); Redis feature reads pipelined.
- **Scalability:** stateless scoring service scales horizontally; feature updates idempotent; generator can replay at 10× speed.
- **Reliability/Fault tolerance:** if Redis misses, scoring degrades to model-with-defaults + conservative rules (documented degraded mode, flagged in response); outbox pattern for audit events.
- **Security:** strict RBAC, PII field-level encryption at rest, no PII in logs, immutable audit trail.
- **Availability:** health/readiness probes, compose restart policies, graceful shutdown draining in-flight scores.
- **Maintainability:** feature definitions as versioned code; model cards per release; ADRs.
- **Monitoring:** decision latency, score distribution drift (PSI), label delay tracking, alert precision by cohort.

9. System Architecture



10. Complete Folder Structure

```

aegisflow/
├─ README.md
├─ docker-compose.yml          # postgres, redis, qdrant, mlflow, services, frontend
├─ Makefile                    # up/seed/generate/train/test/loadtest
├─ .env.example
├─ .pre-commit-config.yaml
├─ .github/workflows/
│   └─ ci.yml                  # lint, tests, build
│   └─ retrain.yml             # scheduled retrain + champion/challenger report
├─ dvc.yaml                    # dataset prep + training stages
├─ params.yaml
├─ data/                       # DVC-tracked; iieee-cis + generated synthetic
├─ docs/
│   └─ architecture.md
│   └─ typologies.md          # documented laundering patterns the generator injects
│   └─ adr/0001-feature-store.md
│   └─ adr/0002-multi-agent-design.md
│   └─ runbook.md
├─ generator/
│   └─ Dockerfile
│   └─ pyproject.toml
│   └─ src/txgen/
│       └─ __init__.py
│       └─ main.py             # CLI: seed entities, stream transactions at rate R
│       └─ config.py
│       └─ population.py       # accounts, devices, merchants, IPs with realistic dists
│       └─ behavior.py         # per-account spending behavior models
│       └─ fraud_rings.py      # coordinated fraud ring injection
│       └─ typologies.py       # smurfing, layering, round-tripping generators
│       └─ labels.py           # ground-truth label sink for evaluation
├─ services/
│   └─ ingestion/
│       └─ Dockerfile
│       └─ pyproject.toml
│       └─ src/ingestion/
│           └─ main.py          # FastAPI app
│           └─ config.py
│           └─ api/routes.py    # POST /transactions, entity upserts
│           └─ schemas.py       # Transaction, Entity pydantic models
│           └─ persistence.py   # SQLAlchemy writers + edge upserts
│           └─ online_features.py # atomic Redis updates (LUA scripts for windows)
│   └─ scoring/
│       └─ Dockerfile
│       └─ pyproject.toml
│       └─ src/scoring/
│           └─ main.py

```

```

| | | └─ config.py
| | | └─ api/routes.py      # POST /score, GET /thresholds, PUT /thresholds
| | | └─ schemas.py        # ScoreRequest, Decision, ReasonCode
| | | └─ feature_reader.py  # pipelined Redis reads, default fallbacks
| | | └─ rules.py           # deterministic rule layer + rule registry
| | | └─ model_runtime.py   # champion+challenger load from MLflow, hot swap
| | | └─ explain.py        # TreeSHAP top-k reason codes (precomputed explainer)
| | | └─ decision.py       # score+rules → approve/review/decline, cost logic
| | | └─ shadow.py         # challenger logging + disagreement metrics
| | └─ casework/
| | | └─ Dockerfile
| | | └─ pyproject.toml
| | | └─ src/casework/
| | | | └─ main.py          # FastAPI: cases, alerts, SAR endpoints
| | | | └─ config.py
| | | | └─ api/
| | | | | └─ cases.py       # queue, detail, assign, disposition
| | | | | └─ sar.py         # draft, edit, approve, export
| | | | | └─ auth.py        # login, JWT issuance
| | | | └─ schemas.py
| | | | └─ correlation.py   # alert→case grouping (entity/ring clustering)
| | | | └─ priority.py      # case priority scoring, SLA timers
| | | | └─ audit.py         # append-only audit writer (outbox consumer)
| | | | └─ export/fincen.py # SAR export to structured format
| | └─ investigator/
| | | └─ Dockerfile
| | | └─ pyproject.toml
| | | └─ src/investigator/
| | | | └─ main.py          # FastAPI: POST /cases/{id}/investigate (async job)
| | | | └─ config.py
| | | | └─ orchestrator/
| | | | | └─ graph.py       # LangGraph: supervisor + 5 agents, state, edges
| | | | | └─ state.py       # CaseState: evidence, findings, draft, qa_verdict
| | | | | └─ prompts.py    # per-agent system prompts + SAR template
| | | | └─ agents/
| | | | | └─ link_analyst.py # tools: entity graph queries, ring detection
| | | | | └─ pattern_analyst.py # tools: txn timeline, typology matchers
| | | | | └─ media_researcher.py # RAG over bundled news corpus (Qdrant)
| | | | | └─ narrative_writer.py # drafts SAR narrative from typed findings
| | | | | └─ qa_reviewer.py # checks narrative claims vs evidence pack
| | | | └─ tools/
| | | | | └─ graph_tools.py # k-hop neighborhood, path-to-fraud, components
| | | | | └─ txn_tools.py  # aggregates, timelines, velocity summaries
| | | | | └─ typology_tools.py # deterministic detectors (structuring windows...)
| | | | └─ rag/
| | | | | └─ ingest_news.py # chunk+embed synthetic adverse-media corpus
| | | | | └─ retriever.py  # hybrid search + entity-name filtering

```

```

|     └─ evaluation/
|         └─ golden_cases.jsonl
|         └─ evaluate.py      # evidence recall, narrative faithfulness scores
└─ ml/
    └─ pyproject.toml
    └─ src/aegis_ml/
        └─ features/
            └─ definitions.py    # SINGLE SOURCE: feature specs used online+offline
            └─ offline.py       # point-in-time batch computation (PyArrow)
            └─ graph_features.py # NetworkX jobs → per-entity graph features
        └─ training/
            └─ prepare_ieee.py   # IEEE-CIS prep as DVC stage
            └─ train_fraud.py    # XGBoost + LightGBM, calibration, MLflow
            └─ registry.py       # champion/challenger promotion policy
        └─ evaluation/
            └─ metrics.py        # recall@FPR, PR-AUC, cost curves, cohort slices
            └─ fairness.py       # disparate impact checks across cohorts
        └─ monitoring/
            └─ drift.py          # PSI on features + scores
            └─ label_feedback.py # disposition labels → retraining dataset
└─ frontend/
    └─ Dockerfile
    └─ package.json
    └─ vite.config.ts
    └─ src/
        └─ main.tsx / App.tsx / api/client.ts
        └─ pages/
            └─ LiveDecisions.tsx # streaming decision feed + latency stats
            └─ CaseQueue.tsx     # SLA-sorted queue, filters, assignment
            └─ CaseWorkspace.tsx # graph view, timeline, evidence, narrative editor
            └─ ModelOps.tsx      # champion/challenger, drift, thresholds
        └─ components/
            └─ EntityGraph.tsx   # force-directed graph (d3/react-force-graph)
            └─ TxnTimeline.tsx
            └─ ReasonCodes.tsx
            └─ NarrativeEditor.tsx # tracked-changes SAR editor
└─ db/
    └─ migrations/...          # alembic
    └─ seed/seed.py           # users, rules, sanctions list, news corpus load
└─ tests/
    └─ unit/                  # features, rules, decision logic, correlation
    └─ integration/          # feature-store parity, registry, graph jobs
    └─ api/                  # contract + RBAC matrix tests
    └─ load/locustfile.py     # 300 TPS scoring profile, p95 assertion
    └─ e2e/test_fraud_to_sar.py # inject ring → decline → case → draft → approve

```

11. File-by-File Explanation (by module)

- **generator/** — `population.py` builds a realistic entity universe (log-normal balances, device sharing); `typologies.py` injects labeled laundering patterns with tunable difficulty; `labels.py` writes ground truth to a separate table the models never see, only evaluation does.
- **ml/features/definitions.py** — the project's most important file: each feature declared once (name, entity keys, window, aggregation, default). `offline.py` compiles definitions into point-in-time batch queries; `services/ingestion/online_features.py` compiles the *same* definitions into Redis update operations. Unit tests assert online/offline parity on replayed streams.
- **scoring/** — `feature_reader.py` does one pipelined Redis round-trip; `rules.py` is a registry of pure-function rules with IDs (auditable); `explain.py` precomputes the TreeSHAP explainer at model load so per-request explanation costs ~1 ms; `decision.py` maps calibrated probability + rule hits + cost matrix to a decision.
- **casework/** — `correlation.py` clusters alerts sharing entities/ring IDs into cases; `audit.py` consumes the transactional outbox so audit records can't be lost or forged; `export/fincen.py` renders approved narratives into a structured SAR-like schema.
- **investigator/** — `orchestrator/graph.py` wires a supervisor pattern (see §15); `tools/` are deterministic, tested Python functions — agents *interpret*, tools *compute*; `evaluation/evaluate.py` scores agent output against golden cases (did it find the injected ring? is every narrative claim traceable?).
- **frontend/** — React SPA; `EntityGraph.tsx` renders the k-hop neighborhood with risk-colored nodes; `NarrativeEditor.tsx` shows the draft with citation chips linking each paragraph to evidence items.
- **db/** — Alembic migrations own all schema; `seed.py` loads demo users, the mock sanctions list, and the adverse-media corpus.

12. Database Design

PostgreSQL

- `accounts`: `id` UUID PK, `holder_name`, `opened_at`, `risk_tier` ENUM, `kyc_level` SMALLINT, `status`. Index on `risk_tier`.
- `devices` / `ips` / `merchants`: `id` UUID PK, natural attrs (`fingerprint`, `ip` INET, `mcc`), `first_seen`, `reputation` REAL.
- `entity_edges`: `id` BIGSERIAL, `src_type`, `src_id`, `dst_type`, `dst_id`, `edge_type` VARCHAR(32), `first_seen`, `weight` REAL. Indexes (`src_type,src_id`), (`dst_type,dst_id`) — the graph store.
- `transactions`: `id` UUID PK, `ts` TIMESTAMPTZ, `account_id` FK, `counterparty_id`, `merchant_id` FK NULL, `device_id` FK NULL, `ip_id` FK NULL, `amount` NUMERIC(14,2), `currency` CHAR(3), `channel` ENUM(card,transfer,wallet), `status`. Indexes (`account_id`, `ts` DESC), (`ts`) (BRIN).
- `decisions`: `id` UUID PK, `transaction_id` FK UNIQUE, `score` REAL, `calibrated_prob` REAL, `decision` ENUM(approve,review,decline), `reason_codes` JSONB, `rule_hits` JSONB, `model_version`, `latency_ms` REAL, `degraded` BOOL, `ts`. Index (`decision`, `ts` DESC).
- `alerts`: `id` UUID PK, `transaction_id` FK, `alert_type`, `severity`, `case_id` FK NULL, `created_at`.
- `cases`: `id` UUID PK, `title`, `priority` REAL, `status` ENUM(open,investigating,pending_review,closed), `assigned_to` FK users NULL, `sla_due` TIMESTAMPTZ, `disposition` ENUM(fraud,not_fraud,sar_filed) NULL, `created_at`.

- `evidence_items`: `id` UUID PK, `case_id` FK, `kind` ENUM(graph, timeline, typology, media, note), `payload` JSONB, `source_ref` JSONB, `created_by` ENUM(agent, human), `created_at`.
- `sar_drafts`: `id` UUID PK, `case_id` FK, `version` INT, `narrative_md` TEXT, `citations` JSONB, `status` ENUM(draft, edited, approved, rejected), `author` ENUM(agent, analyst), `reviewed_by` FK NULL, `created_at` UNIQUE (`case_id`, `version`).
- `users`, `audit_log`, `ground_truth_labels` (generator-only, excluded from serving), `sanctions_list`.

Redis (online features) — hashes `feat:acct:{id}` (velocity counters, profile aggregates), sorted sets for sliding windows `vel:acct:{id}:1h`, graph cache `gfeat:acct:{id}` (component size, dist-to-fraud, risk rank), TTL-managed.

Qdrant — collection `adverse_media`: chunk vectors, payload `{article_id, entities[], date, source, text}`.

ER summary: accounts/devices/ips/merchants are vertices connected N–N through `entity_edges` (the graph); transactions N–1 to each entity type; transactions 1–1 `decisions`; `decisions` 1–N `alerts`; `alerts` N–1 `cases`; `cases` 1–N `evidence_items` and 1–N versioned `sar_drafts`; `users` 1–N case assignments, reviews, and `audit_log` entries.

13. Data Flow

1. Generator streams transactions → ingestion validates → PostgreSQL insert + atomic Redis feature updates (LUA) + edge upserts.
2. Ingestion calls scoring synchronously (or scoring is called directly by the "payment gateway" path): scoring reads Redis features in one pipeline, applies rules, runs champion (+ challenger shadow), computes SHAP reasons, returns decision; decision + audit event persisted.
3. `review` / `decline` decisions and typology detector hits create alerts; correlation groups them into cases; priority + SLA assigned.
4. Analyst (or auto-trigger for high priority) launches investigation → orchestrator runs the agent team → evidence items + SAR draft v1 persisted.
5. Analyst reviews in workspace, edits (v2...), approves → export; disposition recorded.
6. Dispositions become labels; nightly job assembles a point-in-time-correct training set; scheduled retrain produces a challenger; champion/challenger report reviewed before promotion.
7. Drift jobs monitor feature/score distributions; graph job refreshes graph features hourly.

Scenario walkthrough: 03:40 — fourteen card transactions fire across six accounts that share one device fingerprint. Scoring declines nine of them in under 100 ms each, reason codes attached (velocity spike, device sharing degree, one hop from a known-fraud account). Correlation recognizes the shared entities and opens a single ring case instead of fourteen tickets. By 03:47 the agent team has assembled the entity graph, the transaction timeline, a matched smurfing typology, and one adverse-media hit; the narrative writer drafts the SAR. Analyst Priya reviews the evidence-linked draft at 09:15 and approves the filing in 12 minutes — work that previously took an afternoon.

14. AI Pipeline

- **Ingestion/validation:** pydantic schemas, currency/amount sanity, entity referential checks.

- **Feature engineering:** velocity windows (1m/1h/24h/7d), behavioral z-scores vs. account profile, device/IP sharing degrees, graph features (component size, shortest-path-to-labeled-fraud, propagated risk), time-of-day/geo deltas. Point-in-time correctness enforced in offline builds (features as-of transaction time only).
- **Training:** XGBoost champion + LightGBM alternative on IEEE-CIS + synthetic; stratified time-based splits (train on weeks 1–6, test 7–8 — never random shuffle); isotonic calibration; class imbalance via `scale_pos_weight` + threshold-by-cost.
- **Evaluation:** recall @ 1% FPR (headline), PR-AUC, cost curves, cohort slices (amount bands, channels), fairness screens; model card generated per run.
- **Deployment:** MLflow registry, champion/challenger with shadow logging; promotion requires challenger lift $\geq$ X% on rolling labeled window.
- **Monitoring/feedback:** PSI drift, latency SLOs, disagreement rate, label-delay-aware performance tracking; analyst dispositions close the loop.

15. Agentic AI Workflow

Supervisor + 5 specialists (LangGraph), all outputs typed and evidence-linked:

1. **Supervisor** — plans the investigation from case metadata, dispatches specialists, decides sufficiency, max 12 total agent turns.
2. **Link Analyst** — tools: `khop_neighborhood`, `path_to_known_fraud`, `shared_device_report`. Output: `RingFinding` objects (entities, edges, confidence).
3. **Pattern Analyst** — tools: `txn_timeline`, `velocity_summary`, `typology_scan` (deterministic detectors for structuring/layering). Output: `PatternFinding` with matched typology + supporting transaction IDs.
4. **Adverse-Media Researcher** — hybrid RAG over the news corpus filtered by case entities; output: media excerpts with article citations (or explicit "no adverse media found").
5. **Narrative Writer** — consumes only the typed findings (never raw model access to the DB) and drafts the SAR narrative in FinCEN style: who/what/when/where/why/how, every paragraph tagged with evidence IDs.
6. **QA Reviewer** — adversarial pass: verifies every narrative claim maps to an evidence item, checks amounts/dates against `txn_tools`, flags speculation. Fail → one revision loop → else escalate to human with QA notes.

Memory: case state is the shared memory (persisted, resumable); closed-case summaries embedded for "similar past cases" retrieval. **Recovery:** per-agent timeout + retry; a failed specialist yields a typed `FindingUnavailable` so the narrative writer discloses gaps instead of inventing. **HITL:** nothing leaves `draft` status without analyst approval; all agent actions audited.

MCP interface: the investigation tools (`khop_neighborhood`, `txn_timeline`, `typology_scan`, adverse-media search) are additionally exposed through a **Model Context Protocol (MCP) server**, letting vetted external LLM clients and future compliance agents call them under the same RBAC, PII-masking, and audit-trail guarantees — tool reuse without duplicating the security layer.

16. Generative AI Workflow

- **Retrieval:** adverse-media corpus (synthetic, generated once and versioned) chunked per-article-section, embedded with `bge-small-en-v1.5`, hybrid dense+sparse search with entity-name payload filters, top-8 → cross-encoder rerank → top-3.
- **Context:** narrative writer receives a structured findings JSON + SAR template — deliberately *not* free-form retrieval, which keeps hallucination surface minimal.
- **Hallucination mitigation:** three layers — (1) writer restricted to typed findings, (2) QA reviewer claim-checks against tools, (3) UI shows citation chips; uncited paragraphs render with a warning badge and block approval.
- **Evaluation:** golden cases with known injected rings: evidence recall (did agents surface the ring members?), narrative faithfulness (claim-level audit), typology identification accuracy; run in CI for `investigator/` changes.
- **Prompt flow:** all prompts versioned in `prompts.py` with changelog; prompt changes require golden-case eval pass.

17. APIs

Endpoint	Method	Request	Response	Auth
<code>/api/v1/auth/login</code>	POST	<code>{email,password}</code>	JWT	none
<code>/api/v1/transactions</code>	POST	transaction body	<code>{id, decision}</code>	service key
<code>/api/v1/score</code>	POST	<code>ScoreRequest</code>	<code>{decision, prob, reason_codes[], rule_hits[], latency_ms}</code>	service key
<code>/api/v1/thresholds</code>	GET/PUT	— / <code>{review_t, decline_t}</code>	current config	manager
<code>/api/v1/cases</code>	GET	<code>?status=&assigned=&sort=sla</code>	paginated queue	aml_analyst+
<code>/api/v1/cases/{id}</code>	GET	—	case + evidence + drafts	aml_analyst+
<code>/api/v1/cases/{id}/investigate</code>	POST	<code>{}</code>	<code>{job_id}</code> (async)	aml_analyst+
<code>/api/v1/cases/{id}/graph</code>	GET	<code>?hops=2</code>	nodes/edges for viz	aml_analyst+
<code>/api/v1/cases/{id}/disposition</code>	POST	<code>{disposition, rationale}</code>	updated case	aml_analyst+
<code>/api/v1/sar/{case_id}/draft</code>	GET/PUT	— / edited narrative	versioned draft	aml_analyst+
<code>/api/v1/sar/{case_id}/approve</code>	POST	<code>{rationale}</code>	approved + export ref	manager
<code>/api/v1/models /</code> <code>/api/v1/drift</code>	GET	—	registry + drift reports	ds/admin
<code>/health</code> , <code>/metrics</code>	GET	—	probes / Prometheus	none

Every listed resource also exposes the standard REST variants (GET by id, PUT, DELETE where the role permits); the complete, always-current contract is the auto-generated OpenAPI spec served at `/docs` on

each service.

18. Security Design

- **AuthN/AuthZ:** JWT with role + case-team claims; scoring path uses rotating service keys; manager-only threshold and approval endpoints.
- **PII protection:** holder names/PII columns encrypted at rest (pgcrypto), masked by default in API responses (`␣*** ␣**`) unless role permits; logs scrubbed by a logging filter; LLM prompts receive pseudonymous entity IDs, never raw PII — a mapping layer re-inserts display names only in the UI.
- **Audit & integrity:** append-only audit table via outbox; SAR drafts immutable per version.
- **Input validation:** pydantic, parameterized SQL, rate limiting on auth endpoints; prompt-injection defense — retrieved media wrapped as untrusted, agents cannot execute free-form queries (tool whitelist).
- **Secrets:** env-based, rotated service keys, GitHub Actions secrets in CI.

19. Testing Strategy

- **Unit:** feature definitions (online/offline parity fixtures), rules, decision cost logic, correlation clustering, typology detectors (each with positive/negative fixtures), SHAP reason mapping.
- **Integration:** ingestion→Redis atomicity (LUA), registry promote/rollback, graph feature job, outbox audit delivery — via testcontainers.
- **API:** contract + full RBAC matrix (every endpoint × role) + PII masking tests.
- **AI eval:** model gates (recall@1%FPR floor) in retrain workflow; agent golden-case suite (evidence recall ≥ 0.8, faithfulness = 1.0 required).
- **Performance:** Locust 300 TPS, assert p95 < 100 ms; Redis-miss degraded-mode chaos test.
- **E2E:** inject smurfing ring → alerts → case → investigation → draft → approve → export.

20. Deployment Strategy

Local: `make up` && `make seed` && `make generate` — full stack with live decision feed in ~2 minutes. CI: lint → unit → integration → build; images to GHCR. Retraining: scheduled workflow builds challenger, publishes comparison report; human promotes via ModelOps page (audited). Prod-like: compose on a VM behind Caddy TLS; documented AWS mapping (RDS, ElastiCache, ECS, OpenSearch) without requiring cloud spend. Rollback: model registry stage revert + threshold rollback are both one-click and audited.

21. Skills Learned

Online/offline feature stores and training-serving skew prevention; sub-100ms ML serving; SHAP explainability; model calibration; cost-sensitive decisioning; champion/challenger MLOps; graph feature engineering (NetworkX); synthetic data generation with labeled typologies; multi-agent LangGraph systems with typed state and adversarial QA; RAG with entity filtering; React + FastAPI full-stack; RBAC and field-level encryption; audit-grade system design; load testing to latency SLOs.

22. Key Technical Achievements

- "Built a real-time fraud engine scoring 300 TPS at p95 < 100 ms with an online/offline feature store guaranteeing training-serving parity."
- "Improved recall 30%+ at 1% FPR over a rules baseline on IEEE-CIS; every decision explained with SHAP reason codes."
- "Designed a 6-agent AML investigation system that assembles evidence packs and drafts SAR narratives with claim-level citations and human approval."
- "Implemented champion/challenger deployment with shadow scoring and audited promotions via MLflow."

23. Standout Engineering Capabilities

1. **Hard latency engineering** — a measured p95 latency budget with pipelined Redis, precomputed explainers, and a degraded mode is production thinking rarely seen outside senior engineering teams.
2. **Feature store built from first principles** — single-source feature definitions compiled to both batch and streaming paths.
3. **Graph-aware fraud detection** — catches rings that per-transaction models miss; great interview story.
4. **Multi-agent system with an adversarial QA agent and zero-autonomy filing policy** — mature, compliance-grade agentic design.
5. **Regulator-style artifacts** — SAR narratives, model cards, immutable audit — shows understanding of *why* explainability matters, not just how.

24. Future Enhancements

Kafka streaming backbone; GNN-based (GraphSAGE) entity risk model; real sanctions/PEP list integration; device-fingerprint SDK; active learning for label-efficient retraining; multi-tenant deployment; case-outcome forecasting; Temporal/Airflow orchestration of retraining.

Appendix — Key Concepts Explained

- **Online/offline feature store** — features defined once and computed two ways: incrementally in Redis for millisecond serving, and in batch for training — eliminating training/serving skew, the classic silent killer of production ML.
- **SHAP reason codes** — per-decision attributions showing which features pushed the score up or down; converts a black-box score into an explanation a compliance officer can defend.
- **Isotonic calibration** — post-processing that makes model scores behave like true probabilities, so a 0.9 really means ~90% — required before cost-based decisioning.
- **Recall @ 1% FPR** — the fraction of fraud caught while falsely flagging only 1% of legitimate transactions; the honest metric for imbalanced fraud problems (accuracy is meaningless here).
- **Champion/challenger** — the production model (champion) serves decisions while a candidate (challenger) scores the same traffic in shadow; promotion requires measured lift.
- **Graph features** — per-transaction features derived from the entity network (shared devices, distance to known fraud), exposing rings that per-transaction models cannot see.
- **Laundering typologies** — recognized money-laundering patterns: *smurfing* (many small deposits), *layering* (chains of transfers to obscure origin), *round-tripping* (money returning to its source).

- **SAR (Suspicious Activity Report)** — the regulatory filing describing suspected financial crime; here drafted by agents from typed evidence, always human-approved.
 - **Outbox pattern** — audit events written in the same database transaction as the business change, then relayed — so the audit trail can never silently miss an event.
 - **MCP (Model Context Protocol)** — an open standard for exposing tools to LLM clients; the investigation tools are served over MCP under the same RBAC and audit guarantees.
-

Milestone Plan

Milestone	Deliverables
M1	Repo + compose, schema/migrations, synthetic generator with typologies, IEEE-CIS ETL (DVC)
M2	Feature definitions module, offline builder, online Redis path + parity tests, entity graph + graph features
M3	Fraud models + calibration + MLflow, evaluation suite, champion/challenger registry policy
M4	Scoring service (rules, SHAP, decisions, shadow), load tests to SLO, alert/case engine
M5	Investigator agents + tools + golden-case eval, adverse-media RAG, SAR draft/edit/approve flow
M6	React workbench complete, RBAC/audit/PII hardening, drift + retrain workflow, E2E tests, docs + demo